МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное
учреждение высшего образования
**«Санкт-Петербургский политехнический
университет Петра Великого»**
Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
02.03.01_03 Технологии машинного обучения и искусственного интеллекта

Отчёт по лабораторной работе № 1 по дисциплине «Дискретная математика»

«Операции над мультимножествами и генерация бинарного кода Грея»

Выполнил:
Студент группы 5130201/50302 _____ Калинский В.И.
Принял:
Преподаватель _____ Востров А.В.

«____»_____ 2026 г.

Санкт-Петербург, 2026

Содержание

Введение **4**

1 Математическое описание **6**

1.1 Понятие множества, мультимножества и универсума 6

1.2 Теоретико-множественные и арифметические операции 7

1.3 Алгоритм генерации бинарного кода Грея 9

2 Особенности реализации **10**

2.1 Архитектура программного комплекса и структуры данных . . 10

2.2 Спецификация функций и методов 10

3 Результаты работы **15**

3.1 Сценарий 1. Генерация универсума на основе 3-разрядного кода Грея 15

3.2 Сценарий 2. Тестирование защиты от некорректного пользовательского ввода 16

3.3 Сценарий 3. Заполнение мультимножеств в ручном и автоматическом режимах 16

3.4 Сценарий 4. Отображение исходных мультимножеств (пункт меню 1) 18

3.5 Сценарий 5. Проверка теоретико-множественных операций (пункты меню 2–6) 18

3.6 Сценарий 6. Проверка арифметических операций (пункты меню 7–10) 19

Заключение **21**

Список литературы. **22**

Приложение А. Заголовочный файл MultiSet.hpp **23**

Приложение Б. Файл реализации MultiSet.cpp **25**

Приложение В. Файл точки входа и интерфейса main.cpp **28**

Приложение Г. Скрипт сборки CMakeLists.txt 35

Введение

В современной дискретной математике и информатике классическая теория множеств часто оказывается недостаточной для описания реальных информационных структур. Во многих задачах возникает необходимость оперировать наборами элементов, в которых один и тот же объект может присутствовать многократно. Такие математические объекты называются *мультимножествами*.

Для практического применения мультимножеств в вычислительных системах требуется определить конечное базовое множество(носитель) и универсальное мультимножество(универсум), относительно которого корректно определяются операции на мультимножествах. В качестве эффективного базиса для формирования универсума в данной работе выступает двоичный код Грея.

Цель работы: изучение алгебраических и теоретико-множественных свойств мультимножеств, освоение алгоритмов генерации перестановок и кодов Грея, а также программная реализация консольного приложения для моделирования операций над мультимножествами.

Постановка задачи:

1. Разработать алгоритм и программный модуль генерации n-разрядного бинарного кода Грея, где разрядность $n \in [1, 10]$ задается пользователем. Результирующая последовательность кодовых слов длины $N = 2^n$ служит упорядоченным носителем элементов универсума.

2. Реализовать создание двух мультимножеств A и B над построенным носителем с мощностями, задаваемыми пользователем в диапазоне от 0 до 100.

3. Предусмотреть два альтернативных режима наполнения мультимножеств:

   - ручной ввод пользователем (выбор номера элемента носителя и задание его кратности с контролем непревышения целевой мощности);

- автоматическая генерация кратностей элементов на основе псевдослучайных чисел.

4. Сформировать универсальное мультимножество U как арифметическую сумму исходных множеств ($U = A + B$), что гарантирует достаточность кратности универсума для корректного выполнения всех теоретико-множественных операций.

5. Реализовать вычисление следующих операций над мультимножествами:

   - объединение ($A \cup B$);
   - пересечение ($A \cap B$);
   - разность в обоих направлениях ($A \setminus B$ и $B \setminus A$);
   - симметрическая разность ($A \triangle B$);
   - дополнение до универсума ($\overline{A} = U \setminus A$ и $\overline{B} = U \setminus B$);
   - арифметическая сумма ($A + B$);
   - арифметическая разность ($A - B$ и $B - A$);
   - арифметическое произведение ($A \cdot B$);
   - арифметическое деление ($A : B$ и $B : A$).

6. Реализовать строгую фильтрацию некорректного пользовательского ввода и интерактивное меню управления.

1 Математическое описание

1.1 Понятие множества, мультимножества и универсума

Классическим *множеством* называется совокупность определенных, различимых объектов (элементов), мыслимая как единое целое, причем каждый элемент либо принадлежит множеству, либо не принадлежит ему.

Мультимножеством над базовым множеством $X = \{x_1, x_2, \ldots, x_N\}$ называется пара $\hat{A} = (X, k_A)$, где $k_A : X \to \mathbb{N} \cup \{0\}$ – характеристическая функция кратности, сопоставляющая каждому элементу $x_i \in X$ целое неотрицательное число $\alpha_i \geq 0$, называемое *кратностью* (степенью вхождения) элемента в мультимножество.

Символически мультимножество записывается в виде:

$$\hat{A} = \{x_1^{\alpha_1}, x_2^{\alpha_2}, \ldots, x_N^{\alpha_N}\}, \quad \alpha_i \in [0, k], \tag{1}$$

где $x_i^{\alpha_i}$ обозначает вхождение элемента x_i ровно α_i раз. Если $\alpha_i = 0$, элемент фактически отсутствует в мультимножестве. Пустое мультимножество обозначается вектором нулевых кратностей: $\varnothing = (0, 0, \ldots, 0)$.

Мощностью мультимножества $|\hat{A}|$ называется сумма кратностей всех его элементов:

$$|\hat{A}| = \sum_{i=1}^{N} k_A(x_i) = \sum_{i=1}^{N} \alpha_i. \tag{2}$$

Универсумом мультимножеств U называется фиксированное мультимножество над тем же носителем X:

$$U = \{x_1^{m_1}, x_2^{m_2}, \ldots, x_N^{m_N}\}, \quad m_i \geq 0, \tag{3}$$

ограничивающее сверху максимальную кратность элементов любого рассматриваемого мультимножества: $\forall i \ (\alpha_i \leq m_i)$.

В рамках разработанной программы универсум формируется динамически на основе исходных мультимножеств A и B по правилу арифметической суммы:

$$U = A + B \implies m_i = \alpha_{1,i} + \alpha_{2,i}, \tag{4}$$

что математически гарантирует выполнимость базовых операций над операндами A и B внутри универсума U.

1.2 Теоретико-множественные и арифметические операции

Пусть $A = \{x_i^{\alpha_1}\}$ и $B = \{x_i^{\alpha_2}\}$ – два мультимножества над носителем X, а $U = \{x_i^m\}$ – соответствующий универсум ($m = \alpha_1 + \alpha_2$). Правила вычисления кратностей результирующих мультимножеств определяются следующим образом:

1. **Пересечение ($A \cap B$):** Кратность каждого элемента равна минимальной из кратностей в операндах:

$$k_{A \cap B}(x_i) = \min(\alpha_{1,i}, \alpha_{2,i}). \tag{5}$$

2. **Объединение ($A \cup B$):** Кратность каждого элемента равна максимальной из кратностей в операндах:

$$k_{A \cup B}(x_i) = \max(\alpha_{1,i}, \alpha_{2,i}). \tag{6}$$

3. **Дополнение ($\overline{A}$ и $\overline{B}$):** Дополнение мультимножества вычисляется относительно универсума U:

$$k_{\overline{A}}(x_i) = m_i - \alpha_{1,i}, \quad k_{\overline{B}}(x_i) = m_i - \alpha_{2,i}. \tag{7}$$

4. **Разность ($A \setminus B$ и $B \setminus A$):** В теоретическом конспекте разность определяется через пересечение с дополнением: $A \setminus B = A \cap \overline{B}$. Подставляя формулу дополнения, получаем:

$$k_{A \setminus B}(x_i) = \min(\alpha_{1,i}, m_i - \alpha_{2,i}). \tag{8}$$

   Аналогично, для обратной разности:

$$k_{B \setminus A}(x_i) = \min(\alpha_{2,i}, m_i - \alpha_{1,i}). \tag{9}$$

5. **Симметрическая разность ($A \triangle B$):** Определяется как разность между объединением и пересечением мультимножеств:

$$A \triangle B = (A \cup B) \setminus (A \cap B) = (A \cup B) \cap \overline{(A \cap B)}. \tag{10}$$

Кратность элемента:

$$k_{A \triangle B}(x_i) = \min\Big(\max(\alpha_{1,i}, \alpha_{2,i}),\ m_i - \min(\alpha_{1,i}, \alpha_{2,i})\Big). \tag{11}$$

6. **Арифметическая сумма ($A+B$):** В общем теоретическом случае для произвольного универсума с кратностями m_i кратность суммы ограничивается сверху: $k = \min(\alpha_1 + \alpha_2, m)$. Однако в разработанной программе универсум сам формируется как сумма $A + B$, поэтому $m_i = \alpha_{1,i} + \alpha_{2,i}$. Следовательно, ограничение избыточно, и формула имеет вид:

$$k_{A+B}(x_i) = \alpha_{1,i} + \alpha_{2,i}. \tag{12}$$

7. **Арифметическая разность ($A-B$ и $B-A$):** Кратность представляет собой усеченную разность (не может быть отрицательной):

$$k_{A-B}(x_i) = \max(0, \alpha_{1,i} - \alpha_{2,i}), \tag{13}$$

$$k_{B-A}(x_i) = \max(0, \alpha_{2,i} - \alpha_{1,i}). \tag{14}$$

8. **Арифметическое произведение ($A \cdot B$):** Поэлементное произведение кратностей с обязательным ограничением сверху кратностью универсума U:

$$k_{A \cdot B}(x_i) = \min(\alpha_{1,i} \cdot \alpha_{2,i}, m_i). \tag{15}$$

9. **Арифметическое деление ($A : B$ и $B : A$):** Целочисленное деление кратностей. В соответствии с договоренностью при делении на 0 результирующая кратность принимается равной 0:

$$k_{A/B}(x_i) = \begin{cases} \lfloor \alpha_{1,i}/\alpha_{2,i} \rfloor, & \alpha_{2,i} \neq 0, \\ 0, & \alpha_{2,i} = 0. \end{cases} \tag{16}$$

$$k_{B/A}(x_i) = \begin{cases} \lfloor \alpha_{2,i}/\alpha_{1,i} \rfloor, & \alpha_{1,i} \neq 0, \\ 0, & \alpha_{1,i} = 0. \end{cases} \tag{17}$$

1.3 Алгоритм генерации бинарного кода Грея

Бинарным кодом Грея порядка n называется последовательность всех 2^n двоичных векторов длины n, упорядоченная таким образом, что любые два соседних кода, а также первый и последний, отличаются ровно на один разряд.

Для генерации последовательности кодовых слов используется алгоритм, описанный в фундаментальном учебнике Ф. А. Новикова «Дискретная математика для программистов» [1]. Согласно данному алгоритму, переход от кодового слова B_{i-1} к следующему кодовому слову B_i $(i = 1, 2, \ldots, 2^n - 1)$ осуществляется инверсией бита с индексом $p = Q(i) - 1$, где функция $Q(i)$ возвращает номер младшей единицы в двоичной записи шага i:

$$Q(i) = 1 + \max\{k \in \mathbb{N} \cup \{0\} \mid i \text{ делится на } 2^k\}. \tag{18}$$

Алгоритмически вычисление $Q(i)$ представляет собой подсчет числа делений i на 2 до получения нечетного остатка:

$$\begin{cases} q = 1, \quad j = i; \\ \text{пока } (j \pmod 2 == 0): \quad j = j/2, \quad q = q + 1; \\ \text{возврат } q. \end{cases} \tag{19}$$

Тогда закон формирования кодовой последовательности имеет вид:

$$B_0 = 0, \quad B_i = B_{i-1} \oplus (1 \ll (Q(i) - 1)), \quad i = 1, \ldots, 2^n - 1, \tag{20}$$

где $\oplus$ – побитовая операция XOR, $\ll$ – побитовый сдвиг влево. Вектор сгенерированных целых чисел служит базисом упорядоченного универсума.

2 Особенности реализации

2.1 Архитектура программного комплекса и структуры данных

Программный комплекс спроектирован на языке C++ (стандарт C++20) в соответствии с принципами модульности и разделения ответственности:

- `MultiSet.hpp` – спецификация интерфейса класса мультимножества и внешних операторов;
- `MultiSet.cpp` – реализация операций над мультимножествами с применением алгоритмов стандартной библиотеки шаблонов STL (`std::transform`);
- `main.cpp` – модуль консольного взаимодействия, генерации кода Грея, ввода данных и управления логикой;
- `CMakeLists.txt` – сценарий автоматизированной сборки проекта со статической линковкой библиотеки `mylib` и интеграцией динамического санитайзера памяти AddressSanitizer (`-fsanitize=address -fno-omit-frame-pointer -O1`).

Основной структурой данных для представления мультимножества выступает класс `MultiSet`, являющийся адаптером *std::vector<int>*. Внутреннее состояние объекта описывается вектором кратностей `std::vector<int> counts`.

Длина вектора равна текущей мощности носителя универсума $N = 2^{\text{bitrate}}$ (переменная `U_SIZE`). Индекс ячейки вектора i однозначно сопоставлен i-му кодовому слову кода Грея. Глобальный универсум хранится в переменной `MultiSet multi_universe`.

2.2 Спецификация функций и методов

В таблице 1 приведено исчерпывающее описание всех функций и методов программного комплекса с указанием их входных и выходных параметров.

Таблица 1: Спецификация функций и методов программы

Сигнатура	**Входные параметры**	**Выходные параметры / Результат**
`MultiSet::MultiSet()`	Инициализация параметров программы	Инициализированный объект мультимножества размера `U_SIZE`, заполненный нулями.
`MultiSet::begin()`	Инициализация параметров программы	Итератор `std::vector<int>::iterator` на первый элемент вектора кратностей.
`MultiSet::end()`	Инициализация параметров программы	Итератор `std::vector<int>::iterator` на конец вектора кратностей.
`MultiSet::size()`	Инициализация параметров программы	Целое число типа `int` – размер вектора кратностей.
`int& MultiSet::operator[] (size_t pos)`	Позиция элемента `pos` типа `size_t`	Ссылка `int&` на кратность элемента по индексу `pos`.
`void MultiSet::resize (size_t new_size, const int val)`	Новый размер `new_size`, значение заполнения `val`	Модификация размера внутреннего контейнера.
`int Q(int i)`	Целое число $i \geq 1$ (шаг генерации)	Целое число $q \geq 1$ – номер изменяемого разряда в коде Грея.
`void generateGrayCode(int n, vector<int> &target)`	Разрядность n, ссылка на вектор `target`	Заполнение вектора 2^n кодовыми словами Грея.

Таблица 1: Спецификация функций и методов программы

Сигнатура	**Входные параметры**	**Выходные параметры / Результат**
`void printBinary(int code, int bitrate)`	Число `code`, разрядность `bitrate`	Побитовый вывод числа в консоль.
`void printMultiSet(MultiSet &ms, const vector<int> &multi_universe)`	Ссылка на мультимножество `ms`, вектор универсума `universe`	Форматированный вывод пар вида `'код': кратность`.
`int getValidatedInt(const string &prompt, int min_val, int max_val)`	Приглашение ввода, границы диапазона `min_val`, `max_val`	Целое число типа `int`, прошедшее валидацию диапазона и типа.
`void fillManually(MultiSet &ms, const vector<int> &multi_universe, int bitrate, int target_capacity)`	Ссылка на множество `ms`, вектор универсума, разрядность, целевая мощность	Поэлементное ручное заполнение с контролем лимита мощности).
`void fillAutomatically (MultiSet &ms, const vector<int> &multi_universe, int target_capacity)`	Ссылка на множество `ms`, вектор универсума, целевая мощность	Случайное распределение кратностей по элементам носителя.
`MultiSet uni(MultiSet &A, MultiSet &B)`	Ссылки на мультимножества A и B	Объект `MultiSet` – объединение $A \cup B$ $(\max(a_i, b_i)$.

Таблица 1: Спецификация функций и методов программы

Сигнатура	**Входные параметры**	**Выходные параметры / Результат**
`MultiSet insertion(MultiSet &A, MultiSet &B)`	Ссылки на мультимножества A и B	Объект `MultiSet` – пересечение $A \cap B$ ($\min(a_i, b_i)$.
`MultiSet complement(MultiSet &A)`	Ссылка на мультимножество A	Объект `MultiSet` – дополнение $\overline{A} = U \setminus A$.
`MultiSet difference(MultiSet &A, MultiSet &B)`	Ссылки на мультимножества A и B	Объект `MultiSet` – разность $A \setminus B = A \cap \overline{B}$.
`MultiSet sym_-difference(MultiSet &A, MultiSet &B)`	Ссылки на мультимножества A и B	Объект `MultiSet` – симметрическая разность $(A \cup B) \setminus (A \cap B)$.
`MultiSet operator+(MultiSet &A, MultiSet &B)`	Ссылки на операнды A и B	Объект `MultiSet` – арифметическая сумма $A + B$.
`MultiSet operator-(MultiSet &A, MultiSet &B)`	Ссылки на операнды A и B	Объект `MultiSet` – арифметическая разность $A - B$.
`MultiSet operator*(MultiSet &A, MultiSet &B)`	Ссылки на операнды A и B	Объект `MultiSet` – произведение $A \cdot B$, ограниченное U.
`MultiSet operator/(MultiSet &A, MultiSet &B)`	Ссылки на операнды A и B	Объект `MultiSet` – результат деления $A : B$ (с защитой от нуля.
`int main(int argc, char const *argv[])`	Инициализация параметров программы (`argc`, `argv`)	Код возврата операционной системы `0`.

Полный исходный код каждого файла проекта вынесен в отдельные приложения:

- заголовочный файл `MultiSet.hpp` – в Приложении А (см. листинг 1);
- файл реализации операций `MultiSet.cpp` – в Приложении Б (см. листинг 2);
- файл главного модуля и TUI `main.cpp` – в Приложении В (см. листинг 3);
- скрипт системы сборки `CMakeLists.txt` – в Приложении Г (см. листинг 4).

3 Результаты работы

Для всестороннего тестирования разработанного программного комплекса рассмотрены типовые сценарии работы программы, охватывающие как штатную обработку математических операций, так и проверку отказоустойчивости при некорректном пользовательском вводе.

3.1 Сценарий 1. Генерация универсума на основе 3-разрядного кода Грея

Пользователь запускает программу и задает разрядность кода Грея $n = 3$. Программа генерирует $N = 2^3 = 8$ элементов универсума. На экран выводится упорядоченный список бинарных комбинаций: `000`, `001`, `011`, `010`, `110`, `111`, `101`, `100`. Отчетливо видно свойство кода Грея: каждый следующий вектор отличается от предыдущего ровно в одном разряде.

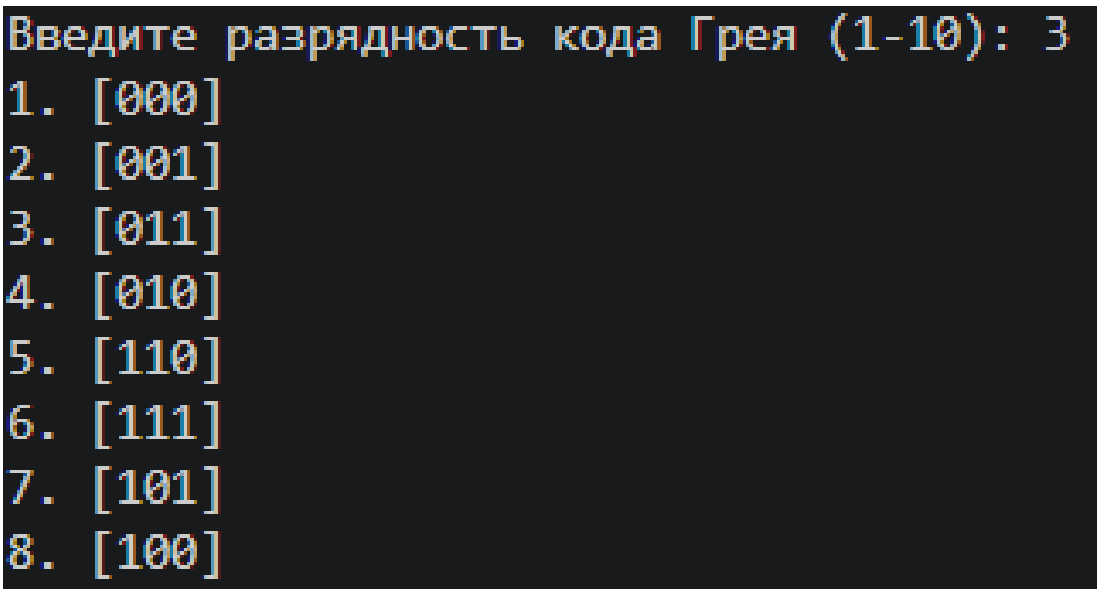


Рис. 1: Генерация 3-разрядного кода Грея и вывод элементов носителя

На рис. 1 продемонстрирован вывод консоли: после ввода числа 3 сформирован упорядоченный список из 8 двоичных чисел универсума с номерами от 1 до 8.

3.2 Сценарий 2. Тестирование защиты от некорректного пользовательского ввода

В рамках сценария выполняется попытка передачи некорректных значений в функцию `getValidatedInt()`:

1. Ввод строковых данных (`"abc"`, `"!?"`) вместо целочисленного значения;
2. Ввод отрицательного числа (-5 при допустимом минимуме 1);
3. Ввод числа, превышающего допустимый диапазон (15 при максимуме 10).

Программа перехватывает сбой потока `cin`, сбрасывает флаг ошибки с помощью `cin.clear()`, очищает входной буфер вызовом `cin.ignore()` и выводит информативное предупреждение с повторным запросом до момента ввода корректного значения.

```
Введите разрядность кода Грея (1-10): abc
Ошибка ввода! Пожалуйста, введите корректное число от 1 до 10.
Введите разрядность кода Грея (1-10): !?
Ошибка ввода! Пожалуйста, введите корректное число от 1 до 10.
Введите разрядность кода Грея (1-10): -5
Ошибка ввода! Пожалуйста, введите корректное число от 1 до 10.
Введите разрядность кода Грея (1-10): 15
Ошибка ввода! Пожалуйста, введите корректное число от 1 до 10.
Введите разрядность кода Грея (1-10):
```

Рис. 2: Отработка защиты от некорректного пользовательского ввода

На рис. 2 показана реакция терминала: при вводе символьной строки и чисел вне диапазона программа сигнализирует об ошибке и не допускает аварийного завершения процесса.

3.3 Сценарий 3. Заполнение мультимножеств в ручном и автоматическом режимах

Задаются параметры: мощность множества A равна 5 (ручной ввод), мощность множества B равна 7 (автоматический ввод). При ручном заполне-

нии множества A пользователь последовательно выбирает элементы носителя и указывает их кратности. При попытке задать кратность, превышающую оставшийся лимит целевой мощности, срабатывает ограничение валидатора. Множество B генерируется автоматически вызовом функции `fillAutomatically(`, которая распределяет суммарную кратность 7 по элементам универсума. Универсум U автоматически формируется как $A + B$.

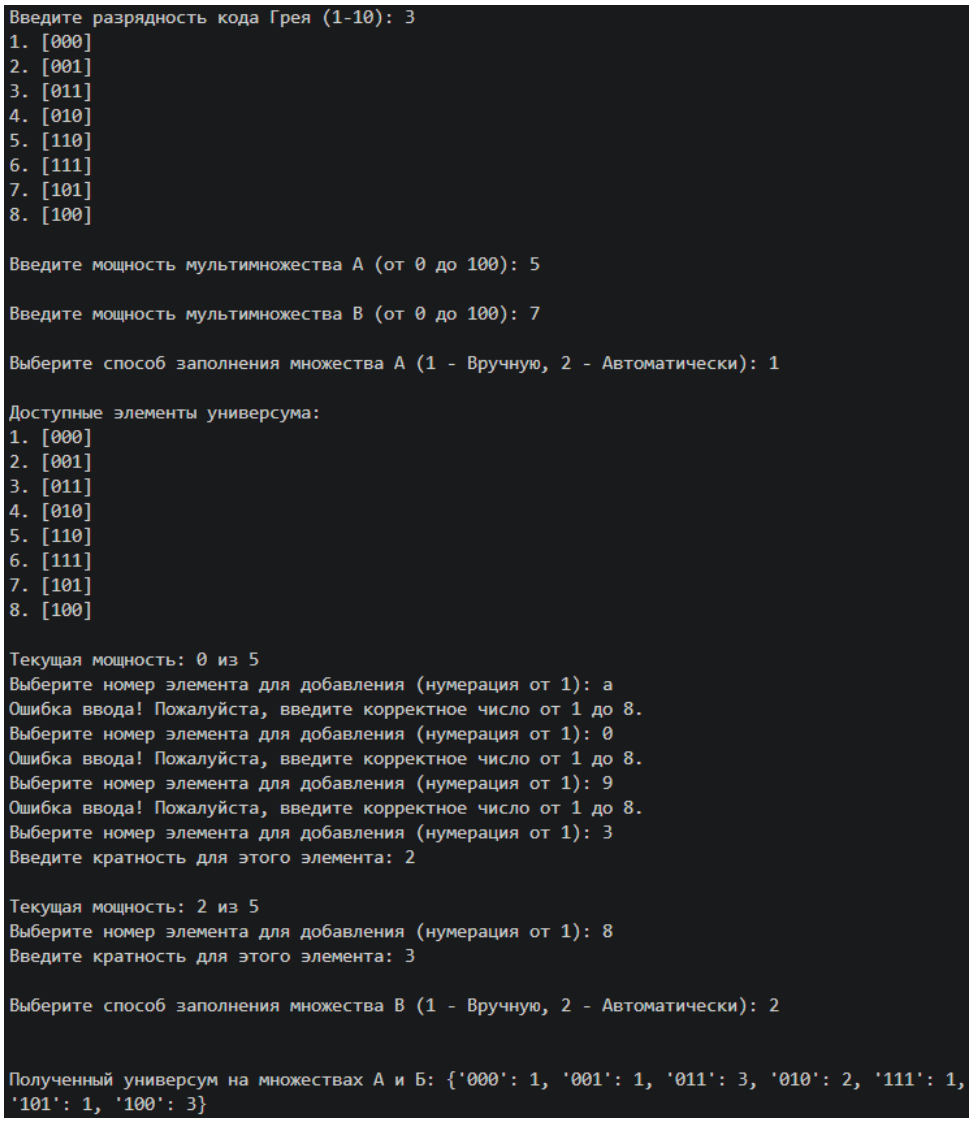


Рис. 3: Ручное заполнение множества A и автоматическая генерация множества B

На рис. 3 представлен протокол формирования множеств: отображаются текущая мощность, запрос выбора элементов, распределение кратностей и результирующее состояние универсума U.

3.4 Сценарий 4. Отображение исходных мультимножеств (пункт меню 1)

В главном меню выбирается пункт 1. Программа отображает сформированные множества A, B и универсум U в формате пар `'код': кратность`:

- Мультимножество A: $\{'011' : 2, '100' : 3\}$;
- Мультимножество B: $\{'000' : 1, '001' : 1, '011' : 1, '010' : 2, '111' : 1, '101' : 1\}$;
- Универсум: $\{'000' : 1, '001' : 1, '011' : 3, '010' : 2, '111' : 1, '101' : 1, '100' : 3\}$.

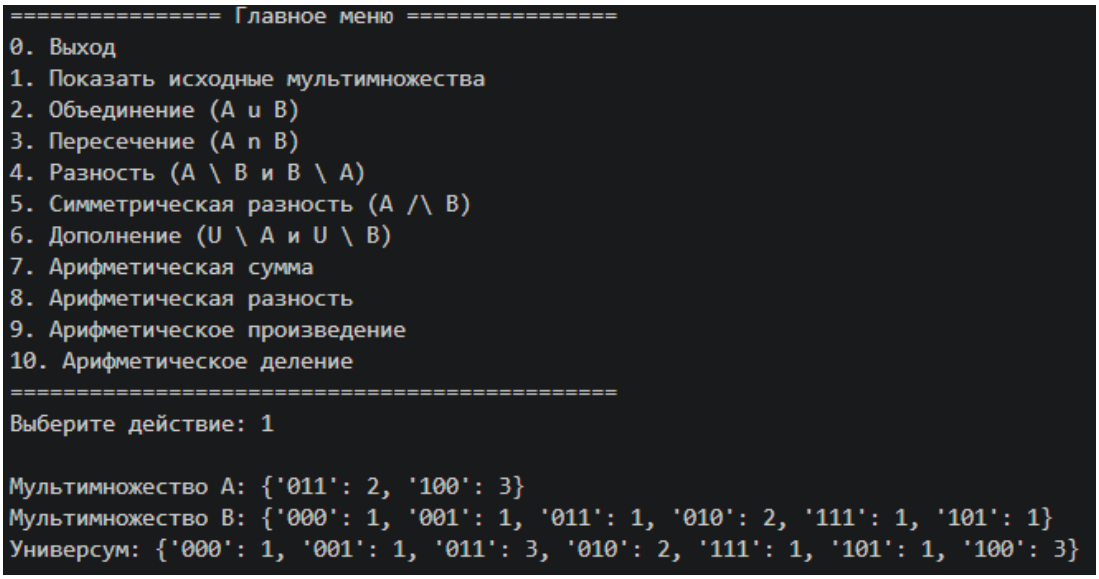


Рис. 4: Вывод содержимого мультимножеств A, B и универсума

На рис. 4 зафиксирован результат работы пункта 1 главного меню: все непустые элементы корректно выведены на экран с указанием их кратностей.

3.5 Сценарий 5. Проверка теоретико-множественных операций (пункты меню 2–6)

После определения мультимножеств в сценарии 5 были выполнены пункты меню 2–6. Получены следующие результаты:

- Объединение $A \cup B$ ($\max(a_i, b_i)$): $\{'000' : 1, '001' : 1, '011' : 2, '010' : 2, '111' : 1, '101' : 1, '100' : 3\}$
- Пересечение $A \cap B$ ($\min(a_i, b_i)$): $\{'011' : 1\}$
- Разность $A \setminus B$ ($\min(a_i, m_i - b_i)$): $\{'011' : 2, '100' : 3\}$
- Разность $B \setminus A$ ($\min(b_i, m_i - a_i)$): $\{'000' : 1, '001' : 1, '011' : 1, '010' : 2, '111' : 1, '101' : 1\}$
- Симметрическая разность $A \triangle B$ ($(A \cup B) \setminus (A \cap B)$): $\{'000' : 1, '001' : 1, '011' : 2, '010' : 2, '111' : 1, '101' : 1, '100' : 3\}$
- Дополнение $\mathcal{U} \setminus A$ ($m_i - a_i$): $\{'000' : 1, '001' : 1, '011' : 1, '010' : 2, '111' : 1, '101' : 1\}$
- Дополнение $\mathcal{U} \setminus B$ ($m_i - b_i$): $\{'011' : 2, '100' : 3\}$

Все полученные программой результаты строго совпадают с ручным теоретическим расчетом.

3.6 Сценарий 6. Проверка арифметических операций (пункты меню 7–10)

После определения мультимножеств в сценарии 5 были выполнены пункты меню 7–10. Получены следующие результаты:

- Арифметическая сумма $A + B$ ($a_i + b_i$): $\{'000' : 1, '001' : 1, '011' : 3, '010' : 2, '111' : 1, '101' : 1, '100' : 3\}$
- Арифметическая разность $A - B$ ($\max(0, a_i - b_i)$): $\{'011' : 1, '100' : 3\}$
- Арифметическая разность $B - A$ ($\max(0, b_i - a_i)$): $\{'000' : 1, '001' : 1, '010' : 2, '111' : 1, '101' : 1\}$
- Арифметическое произведение $A \cdot B$ ($\min(a_i \cdot b_i, m_i)$): $\{'011' : 2\}$
- Арифметическое деление A/B (a_i/b_i): $\{'011' : 2\}$
- Арифметическое деление B/A (b_i/a_i): $\{\}$

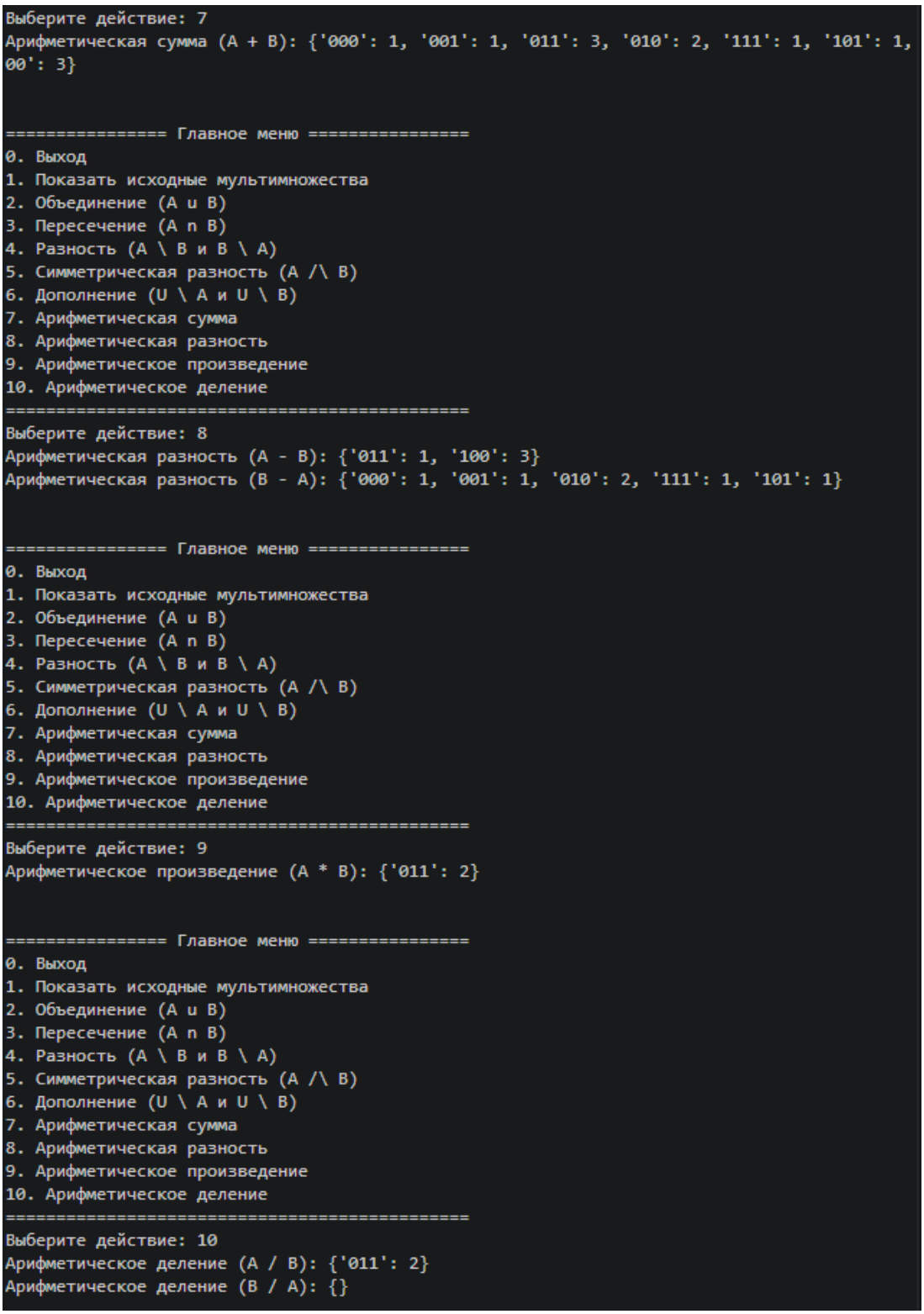


Рис. 5: Результаты выполнения арифметических операций и проверка деления на ноль

На рис. 5 отражен корректный вывод арифметических операций, подтверждающий усечение произведения универсумом и безаварийное деление на ноль.

Заключение

В ходе лабораторной работы на языке C++20 разработан программный комплекс для генерации бинарного кода Грея и выполнения полного набора операций над мультимножествами. В рамках реализации спроектирован класс `MultiSet` на базе вектора кратностей, поддержаны ручной и автоматический режимы ввода, а также динамическое построение универсума $U = A + B$. Реализованы все требуемые теоретико-множественные и арифметические операции, обеспечена строгая валидация пользовательского ввода и проведено тестирование под AddressSanitizer, подтвердившее отсутствие утечек памяти и корректность граничных вычислений.

Достоинства реализованного решения:

- Высокая скорость доступа к элементам и линейная сложность вычисления операций над мультимножеством.
- Лаконичность и безопасность кода благодаря алгоритмам STL (`std::transf`

Недостатки решения:

- Экспоненциальный расход памяти при росте разрядности кода Грея ($N = 2^n$).
- Неэффективность плотного вектора для сильно разреженных мультимножеств.

Масштабирование алгоритма: Для работы с большими универсумами ($n \geq 30$) целесообразно заменить плотный вектор ассоциативным контейнером `unordered_map<int, int>`, реализованный на хэш-таблице и не хранящей нулевых элементов. Также для ускорения поэлементной обработки возможно применение параллельных вычислений.

Список литературы

[1] Новиков Ф. А. Дискретная математика для программистов : Учебник для вузов / Ф. А. Новиков. – 3-е изд. – СПб. : Питер, 2009. – 384 с.

Приложение А. Заголовочный файл MultiSet.hpp

Листинг 1: Заголовочный файл MultiSet.hpp

```
#ifndef SET_HPP
#define SET_HPP

#include <vector>

class MultiSet
{
    std::vector<int> counts;
    public:
    MultiSet();
    std::vector<int>::iterator begin()
    {
        return counts.begin();
    }
    int size()
    {
        return counts.size();
    }
    std::vector<int>::iterator end()
    {
        return counts.end();
    }
    int &operator[](size_t pos)
    {
        return counts[pos];
    }
    void resize(size_t new_size, const int val){
        counts.resize(new_size, val);
    }
};

extern int U_SIZE;
extern MultiSet multi_universe;

MultiSet uni(MultiSet &A, MultiSet &B);
MultiSet insertion(MultiSet &A, MultiSet &B);
MultiSet difference(MultiSet &A, MultiSet &B);
MultiSet sym_difference(MultiSet &A, MultiSet &B);
MultiSet complement(MultiSet &A);

MultiSet operator+(MultiSet &A, MultiSet &B);
MultiSet operator-(MultiSet &A, MultiSet &B);
MultiSet operator*(MultiSet &A, MultiSet &B);
```

```
44        MultiSet operator/(MultiSet &A, MultiSet &B);
45
46        #endif
```

Приложение Б. Файл реализации MultiSet.cpp

Листинг 2: Файл реализации MultiSet.cpp

```
#include "MultiSet.hpp"
#include <algorithm>
#include <cmath>
#include <iostream>

using namespace std;

MultiSet::MultiSet() : counts(U_SIZE, 0) {}

// объединение: max(a, b)
MultiSet uni(MultiSet &A, MultiSet &B)
{
    MultiSet result;
    transform(A.begin(), A.end(), B.begin(), result.begin(), [](int a, int b)
    { return max(a, b); });
    return result;
}

// пересечение: min(a, b)
MultiSet insertion(MultiSet &A, MultiSet &B)
{
    MultiSet result;
    transform(A.begin(), A.end(), B.begin(), result.begin(), [](int a, int b)
    { return min(a, b); });
    return result;
}

// дополнение: m - a, m - кратностьэлта- вуниверсуме
MultiSet complement(MultiSet &A)
{
    MultiSet result;
    transform(A.begin(), A.end(), multi_universe.begin(), result.begin(), [](int a,
        int m)
    { return m - a; });
    return result;
}

// разность: A n -B
MultiSet difference(MultiSet &A, MultiSet &B)
{
    MultiSet notB = complement(B);
    return insertion(A, notB);
}
```

```
43
44      // симметрическаяразность: (A u B) \ (A n B)
45      MultiSet sym_difference(MultiSet &A, MultiSet &B)
46      {
47          MultiSet AuB = uni(A, B);
48          MultiSet AnB = insertion(A, B);
49          return difference(AuB, AnB);
50      }
51
52      // арифметическаясумма
53      MultiSet operator+(MultiSet &A, MultiSet &B)
54      {
55          MultiSet result;
56          // ненужныограничения, укаждогоэлта- универсумакратностьравнасуммекратностейэлов-
                АиБ
57          transform(A.begin(), A.end(), B.begin(), result.begin(), [](int a, int b)
58          { return a + b; });
59          return result;
60      }
61
62      // арифметическаяразность
63      MultiSet operator-(MultiSet &A, MultiSet &B)
64      {
65          MultiSet result;
66          transform(A.begin(), A.end(), B.begin(), result.begin(), [](int a, int b)
67          { return max(0, a - b); });
68          return result;
69      }
70
71      // арифметическоепроизведение
72      MultiSet operator*(MultiSet &A, MultiSet &B)
73      {
74          MultiSet result;
75          for (int i = 0; i < U_SIZE; i++)
76          {
77              result[i] = min(A[i] * B[i], multi_universe[i]);
78          }
79          return result;
80      }
81
82      // арифметическоеделение
83      MultiSet operator/(MultiSet &A, MultiSet &B)
84      {
85          MultiSet result;
86          transform(A.begin(), A.end(), B.begin(), result.begin(), [](int a, int b)
87          { return (b == 0) ? 0 : (a / b); });
```

```
88              return result;
89          }
```

Приложение В. Файл точки входа и интерфейса main.cpp

Листинг 3: Файл точки входа и интерфейса main.cpp

```
#include <string>
#include <limits>
#include <iostream>
#include <vector>
#include <bitset>
#include <format>
#include <algorithm>
#include "MultiSet.hpp"

int U_SIZE = 0;
MultiSet multi_universe;

using namespace std;
int getValidatedInt(const string &prompt = "", int min_val = 0, int max_val =
    numeric_limits<int>::max())
{
    int value;
    while (true)
    {
        cout << prompt;
        if (cin >> value && value >= min_val && value <= max_val)
        {
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            return value;
        }
        cout << "Ошибка ввода! Пожалуйста, введитекорректноечислоот" << min_val << "
            до" << max_val << ".\n";
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }
}

int Q(int i)
{
    int q = 1;
    int j = i;
    while (j % 2 == 0)
    {
        j = j / 2;
        q = q + 1;
    }
    return q;
}
```

```
42
43      void generateGrayCode(int n, vector<int> &target)
44      {
45          target.clear();
46          int B = 0;
47          target.push_back(B);
48
49          int total_steps = 1 << n;
50
51          for (int i = 1; i < total_steps; i++)
52          {
53              int p_index = Q(i) - 1;
54              B ^= 1 << p_index;
55              target.push_back(B);
56          }
57      }
58
59      void printBinary(int code, int bitrate)
60      {
61          for (int i = bitrate - 1; i >= 0; i--)
62          {
63              cout << ((code >> i) & 1);
64          }
65      }
66
67      void printMultiSet(MultiSet &ms, const vector<int> &multi_universe)
68      {
69          if (ms.size() == 0)
70          {
71              cout << "Мультимножество пустое!\n";
72          }
73
74          bool first = true;
75          cout << "{";
76              for (int i = 0; i < multi_universe.size(); ++i)
77              {
78                  if (ms[i] > 0)
79                  {
80                      if (!first)
81                      cout << ", ";
82                      cout << "'" << multi_universe[i] << "': " << ms[i];
83                      first = false;
84                  }
85              }
86              cout << "}\n";
87      }
```

```
88
89      void fillManually(MultiSet &ms, const vector<int> &multi_universe, int bitrate, int
            target_capacity)
90      {
91          int current_capacity = 0;
92
93          cout << "\Доступныеn элементыуниверсума:\n";
94          for (int i = 0; i < multi_universe.size(); ++i)
95          {
96              cout << i + 1 << ". [";
97              printBinary(multi_universe[i], bitrate);
98              cout << "]\n";
99          }
100
101         while (current_capacity < target_capacity)
102         {
103             cout << "\Текущаяn мощность: " << current_capacity << " из" <<
                    target_capacity << "\n";
104             int idx = getValidatedInt("Выберите номерэлементадлядобавлениянумерация( от1)
                    : ", 1, multi_universe.size()) - 1;
105             int count = getValidatedInt("Введите кратностьдляэтогоэлемента: ", 1,
                    target_capacity - current_capacity);
106
107             ms[idx] += count;
108             current_capacity += count;
109         }
110     }
111
112     void fillAutomatically(MultiSet &ms, const vector<int> &multi_universe, int
            target_capacity)
113     {
114         srand(time(nullptr));
115
116         for (int i = 0; i < target_capacity; ++i)
117         {
118             int idx = rand() % target_capacity;
119             ms[idx]++;
120         }
121     }
122
123     int main(int argc, char const *argv[])
124     {
125         int bitrate = getValidatedInt("Введите разрядностькодаГрея(1-10): ", 1, 10);
126         vector<int> universe(1 << bitrate, 0);
127         generateGrayCode(bitrate, universe);
128         U_SIZE = universe.size();
```

```
129
130         for (int i = 0; i < universe.size(); ++i)
131         {
132             cout << i + 1 << ". [";
133             printBinary(universe[i], bitrate);
134             cout << "]\n";
135         }
136
137         int lower_capacity = 0;
138         int upper_capacity = 100;
139         int capacityA = getValidatedInt(format("\Введитеn мощностьмультимножестваA от( {}
                до{}): ", lower_capacity, upper_capacity),
140         lower_capacity, upper_capacity);
141         int capacityB = getValidatedInt(format("\Введитеn мощностьмультимножестваB от( {}
                до{}): ", lower_capacity, upper_capacity), lower_capacity, upper_capacity);
142
143         MultiSet A, B;
144         if (capacityA > 0)
145         {
146             int choiceA = getValidatedInt("\Выберитеn способзаполнениямножестваA (1 -
                Вручную, 2 - Автоматически): ", 1, 2);
147             if (choiceA == 1)
148             fillManually(A, universe, bitrate, capacityA);
149             else
150             fillAutomatically(A, universe, capacityA);
151         }
152         if (capacityB > 0)
153         {
154             int choiceB = getValidatedInt("\Выберитеn способзаполнениямножестваB (1 -
                Вручную, 2 - Автоматически): ", 1, 2);
155             if (choiceB == 1)
156             fillManually(B, universe, bitrate, capacityB);
157             else
158             fillAutomatically(B, universe, capacityB);
159         }
160
161         multi_universe = A + B;
162         cout << "\n\Полученныйn универсумнамножествахАиБ: ";
163         printMultiSet(multi_universe, universe);
164
165         while (true)
166         {
167             cout << "\n\n================ Главноеменю================\n";
168             cout << "0. Выход\n";
169             cout << "1. Показатьисходныемультимножества\n";
170             cout << "2. Объединение(A u B)\n";
```

```
171             cout << "3. Пересечение(A n B)\n";
172             cout << "4. Разность(A \\ B иB \\ A)\n";
173             cout << "5. Симметрическаяразность(A /\\ B)\n";
174             cout << "6. Дополнение(U \\ A иU \\ B)\n";
175             cout << "7. Арифметическаясумма\n";
176             cout << "8. Арифметическаяразность\n";
177             cout << "9. Арифметическоепроизведение\n";
178             cout << "10. Арифметическоеделение\n";
179             cout << "=============================================\n";
180 
181             int menuChoice = getValidatedInt("Выберите действие: ", 0, 10);
182 
183             if (menuChoice == 0)
184             {
185                 cout << "Программа завершена.\n";
186                 break;
187             }
188 
189             MultiSet result;
190 
191             switch (menuChoice)
192             {
193                 case 1:
194                 cout << "\Мультимножествоn A: ";
195                 printMultiSet(A, universe);
196                 cout << "Мультимножество B: ";
197                 printMultiSet(B, universe);
198                 cout << "Универсум: ";
199                 printMultiSet(multi_universe, universe);
200                 break;
201                 case 2:
202                 result = uni(A, B);
203                 cout << "Объединение (A u B): ";
204                 printMultiSet(result, universe);
205                 break;
206                 case 3:
207                 result = insertion(A, B);
208                 cout << "Пересечение (A n B): ";
209                 printMultiSet(result, universe);
210                 break;
211                 case 4:
212                 result = difference(A, B);
213                 cout << "Разность (A \\ B): ";
214                 printMultiSet(result, universe);
215 
216                 result = difference(B, A);
```

```
                cout << "Разность (B \\ A): ";
                printMultiSet(result, universe);
                break;
                case 5:
                result = sym_difference(A, B);
                cout << "Симметрическая разность(A /\\ B): ";
                printMultiSet(result, universe);
                break;
                case 6:
                result = complement(A);
                cout << "Дополнение (U \\ A): ";
                printMultiSet(result, universe);

                result = complement(B);
                cout << "Дополнение (U \\ B): ";
                printMultiSet(result, universe);
                break;
                case 7:
                result = A + B;
                cout << "Арифметическая сумма(A + B): ";
                printMultiSet(result, universe);
                break;
                case 8:
                result = A - B;
                cout << "Арифметическая разность(A - B): ";
                printMultiSet(result, universe);

                result = B - A;
                cout << "Арифметическая разность(B - A): ";
                printMultiSet(result, universe);
                break;
                case 9:
                result = A * B;
                cout << "Арифметическое произведение(A * B): ";
                printMultiSet(result, universe);
                break;
                case 10:
                result = A / B;
                cout << "Арифметическое деление(A / B): ";
                printMultiSet(result, universe);

                result = B / A;
                cout << "Арифметическое деление(B / A): ";
                printMultiSet(result, universe);
                break;
            }
```

```
263             }
264 
265             return 0;
266         }
```

Приложение Г. Скрипт сборки CMakeLists.txt

Листинг 4: Скрипт сборки CMakeLists.txt

```
cmake_minimum_required(VERSION 3.20.0)
set (CMAKE_CXX_STANDARD 20)
project(lab1 LANGUAGES CXX)

set(CMAKE_CXX_FLAGS "-fsanitize=address -fno-omit-frame-pointer -O1")

add_library(mylib STATIC
src/MultiSet.hpp
src/MultiSet.cpp
)

add_executable(main
src/main.cpp
)

target_link_libraries(main mylib)

file(GENERATE
OUTPUT "${CMAKE_CURRENT_BINARY_DIR}/run_test.sh"
CONTENT "cat ../test_data.txt | ./main\n"
)
```